CS2614 — Computer System Architecture

Chapter 4: Register Transfer and Microoperations

Integrated Lecture Notes and Worked Problems

Based on M. Morris Mano, Computer System Architecture, 3rd ed.

1 Register Transfer Language

A digital computer is organized from interconnected hardware modules such as registers, decoders, arithmetic elements, and control logic. The operations performed on information stored in registers are called *microoperations*.

A microoperation is an elementary operation performed on the information stored in one or more registers. The result may replace the previous contents of a register or may be transferred to another register.

WHITEBOARD

---

microoperation = elementary operation on register data

Examples:

transfer, load, clear, increment, shift

A computer's internal organization is described by:

1. registers and their functions

2. microoperations performed

3. control that initiates them

Instead of describing each internal operation in prose, Mano uses a symbolic notation called *register transfer language* (RTL). RTL specifies register transfers, microoperations, and the control functions that initiate them.

WHITEBOARD

RTL = Register Transfer Language

symbolic description of internal computer operations

---

2 Register Transfer

Registers are designated by capital letters or functional abbreviations such as

$$R1, \quad R2, \quad AR, \quad PC, \quad IR.$$

For an n-bit register, bit positions are numbered from 0 at the right through $n-1$ at the left. A portion of a register may be identified explicitly.

WHITEBOARD

$$R(n-1), \ldots, R(1), R(0)$$

For a 16-bit register:

$$\boxed{PC(L) = PC(0\text{–}7)}$$

$$\boxed{PC(H) = PC(8\text{–}15)}$$

The basic register-transfer statement

$$R2 \leftarrow R1$$

means that the contents of $R1$ are copied into $R2$. The previous contents of $R2$ are replaced, while $R1$ is unchanged.

WHITEBOARD

$$\boxed{R2 \leftarrow R1}$$

$$\boxed{R2 \text{ receives the contents of } R1}$$

$$\boxed{R1 \text{ is unchanged}}$$

2.1 Control Functions and Clocked Transfers

A register transfer normally occurs only when a control condition is active. If the control variable is P, the statement

$$P: \quad R2 \leftarrow R1$$

means that the transfer occurs when $P = 1$. The actual loading of the destination register occurs at the active clock transition.

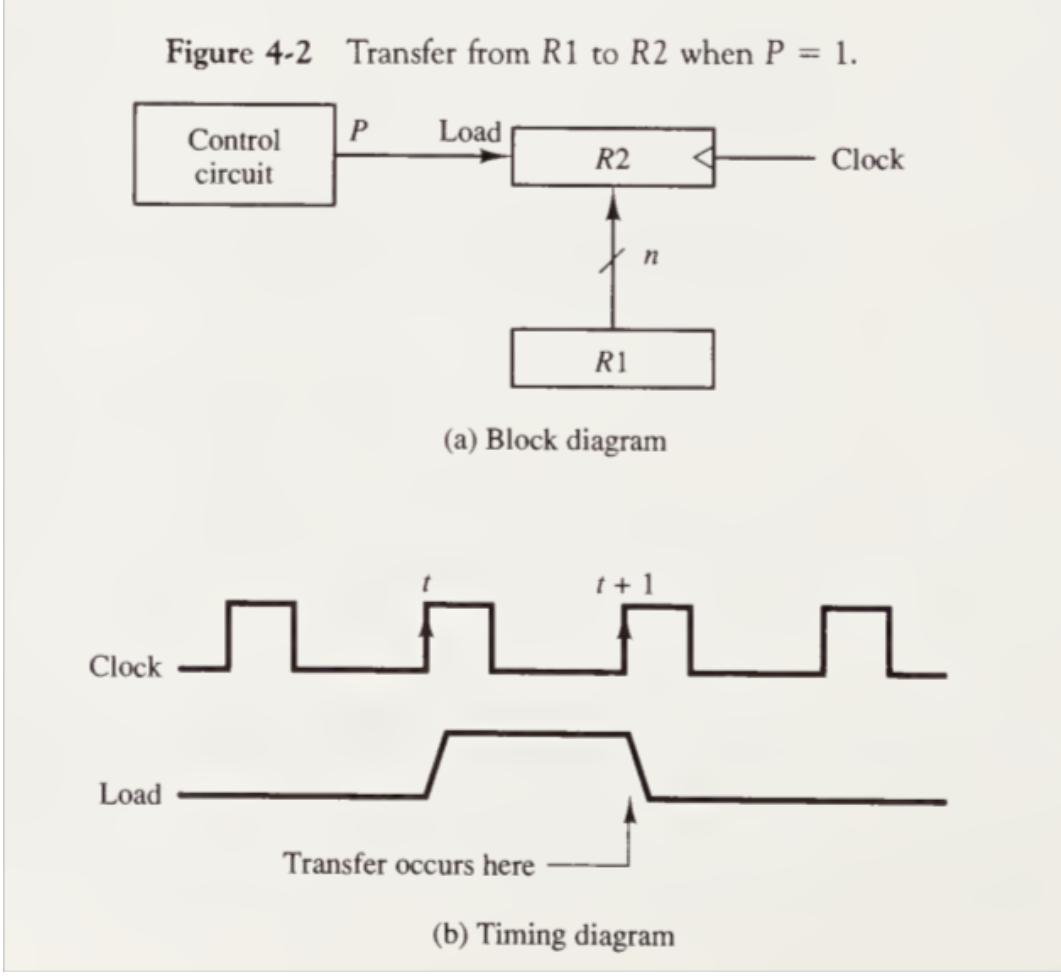
WHITEBOARD

$$\boxed{P: \quad R2 \leftarrow R1}$$

$$P = 1 \Rightarrow \text{transfer enabled}$$

$$\boxed{\text{transfer occurs at the active clock edge}}$$

Corresponds to Mano, Fig. 4-2.



A comma in RTL denotes simultaneous microoperations, provided the hardware can perform them during the same clock transition.

WHITEBOARD

$$\boxed{\text{comma} = \text{simultaneous microoperations}}$$

Example:

$$R2 \leftarrow R1, \qquad R1 \leftarrow R2$$

Worked Example — Textbook Problem 4-1: Simultaneous Register Exchange

Q: Show the block diagram of the hardware that implements

$$yT_2: \qquad R_2 \leftarrow R_1, \qquad R_1 \leftarrow R_2.$$

A: The comma means that both transfers occur on the same clock transition. The two registers are cross-connected, and the control function yT_2 enables both destination load inputs. Because the registers are edge-triggered, each samples the old value of the other at the edge, so no temporary register is required.

WHITEBOARD

$$P = yT_2$$

$$\boxed{R_1^{out} \rightarrow R_2^{in}} \qquad \boxed{R_2^{out} \rightarrow R_1^{in}}$$

$$\boxed{LD(R_1) = LD(R_2) = yT_2}$$

At the active edge:

$$\boxed{R_1^+ = R_2, \qquad R_2^+ = R_1}$$

Worked Example — Register Problem 14: Same Exchange with Control C_3

Q: Implement

$$C_3: \qquad R_2 \leftarrow R_1, \qquad R_1 \leftarrow R_2.$$

A: This is the same simultaneous-exchange structure as Problem 4-1, with C_3 replacing yT_2.

WHITEBOARD

$$\boxed{LD(R_1) = LD(R_2) = C_3}$$

C_3	R_1^+	R_2^+
0	R_1	R_2
1	R_2	R_1

Worked Example — Textbook Problem 4-23: Illegal Simultaneous Assignments

Q: What is wrong with the following RTL statements?

$$\text{(a) } xT: \quad AR \leftarrow \overline{AR}, \qquad AR \leftarrow 0$$

$$\text{(b) } yT: \quad R_1 \leftarrow R_2, \qquad R_1 \leftarrow R_3$$

$$\text{(c) } zT: \quad PC \leftarrow AR, \qquad PC \leftarrow PC + 1$$

A: Each line tries to assign two different next values to the same destination register during one clock transition. A single register cannot load two different values at the same edge. The operations must be separated in time or made mutually exclusive.

WHITEBOARD

$$\boxed{\text{One destination register cannot receive two next values at one edge.}}$$

$$\text{(a) } AR: \ \overline{AR} \text{ or } 0 \quad \text{– not both}$$

$$\text{(b) } R_1: \ R_2 \text{ or } R_3 \quad \text{– not both}$$

$$\text{(c) } PC: \ AR \text{ or } PC + 1 \quad \text{– not both}$$

3 Bus and Memory Transfers

If every register were directly wired to every other register, the interconnection network would become excessive. A common bus provides a shared data path among registers.

WHITEBOARD

$$\boxed{\text{common bus} = \text{shared data path among registers}}$$

For n-bit registers:

$$\boxed{\text{bus width} = n \text{ lines}}$$

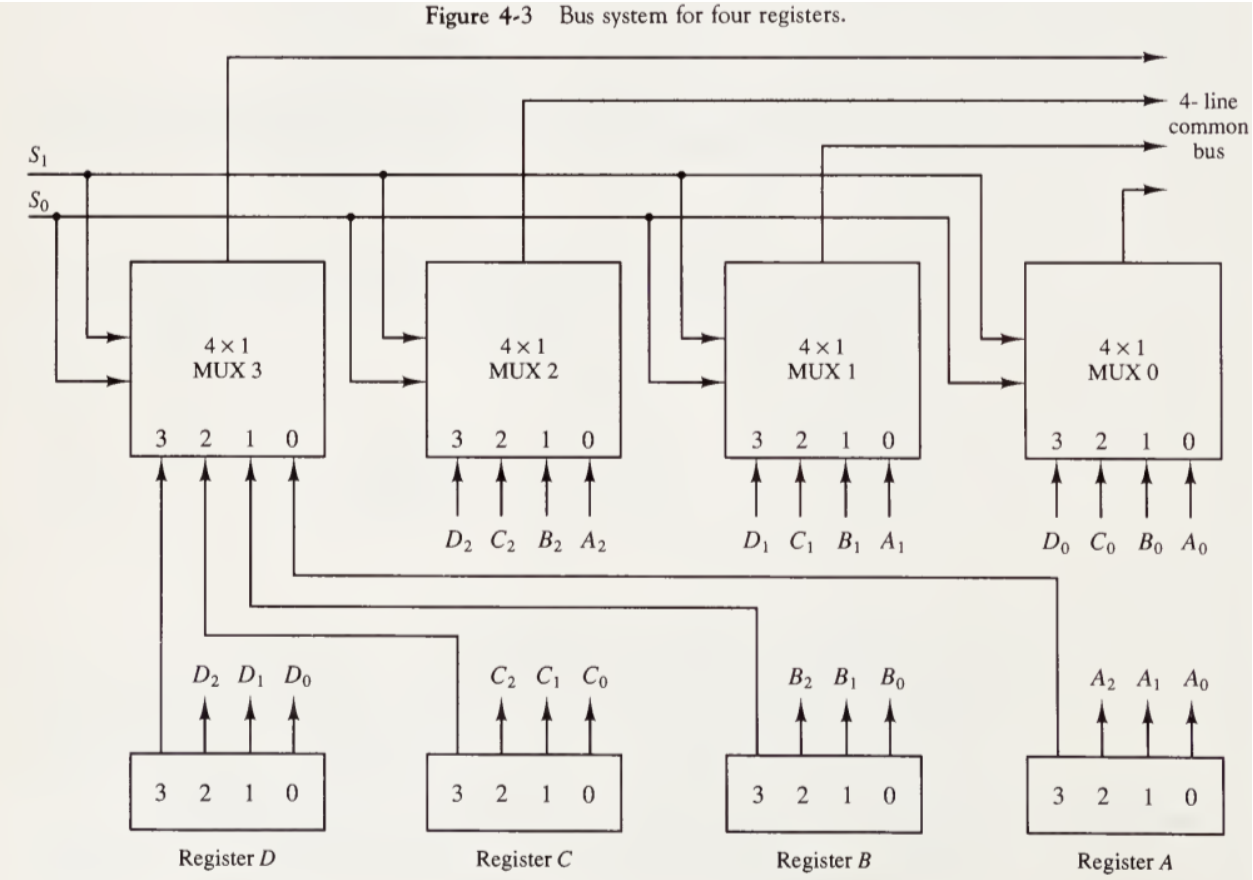
$$\boxed{\text{select source} \rightarrow \text{bus} \rightarrow \text{load destination}}$$

3.1 Multiplexer Bus

Mano's four-register bus uses four 4-to-1 multiplexers, one for each bit position. The same selection inputs choose the source register for all bus bits.

WHITEBOARD

Corresponds to Mano, Fig. 4-3.



S_1	S_0	Register on bus
0	0	A
0	1	B
1	0	C
1	1	D

For k registers of n bits each, the bus requires n multiplexers, each of size k-to-1.

WHITEBOARD

$\boxed{k \text{ registers} \times n \text{ bits}}$

requires

$$\boxed{n \text{ multiplexers, each } k\text{-to-1}}$$

Example:

$$8 \text{ registers} \times 16 \text{ bits} \Rightarrow \boxed{16 \text{ eight-to-one MUXes}}$$

Worked Example — Textbook Problem 4-4: Completing the Bus for Register-to-Register Transfer

Q: What must be added to the bus system of Fig. 4-3 to transfer information from any register to any other register? In particular, provide a path from register C to register A.

A: Fig. 4-3 already selects the source register. To complete a general transfer system, the common bus must also connect to the parallel inputs of every destination register, and every register needs its own load control. To perform $A \leftarrow C$, select C onto the bus and enable A's load input.

WHITEBOARD

$$\boxed{BUS \rightarrow A, B, C, D}$$

For

$$A \leftarrow C :$$

$$\boxed{S_1 S_0 = 10} \qquad \boxed{LD(A) = 1}$$

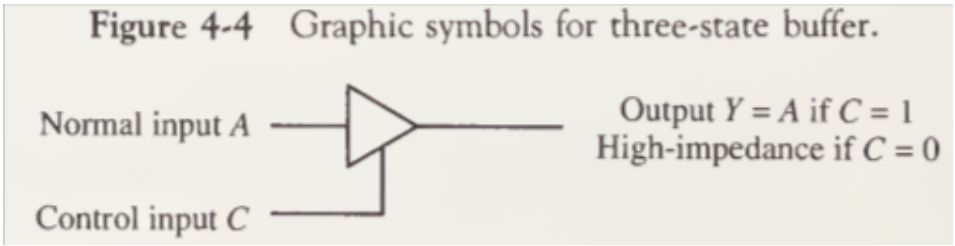
$$BUS \leftarrow C, \qquad A \leftarrow BUS = C$$

3.2 Three-State Bus Buffers

A common bus can also be constructed with three-state buffers. A three-state output can assume logic 0, logic 1, or the high-impedance state Z. High impedance behaves like an open circuit, allowing multiple outputs to share one physical bus line safely.

WHITEBOARD

Corresponds to Mano, Fig. 4-4.



$$\boxed{0, \quad 1, \quad Z}$$

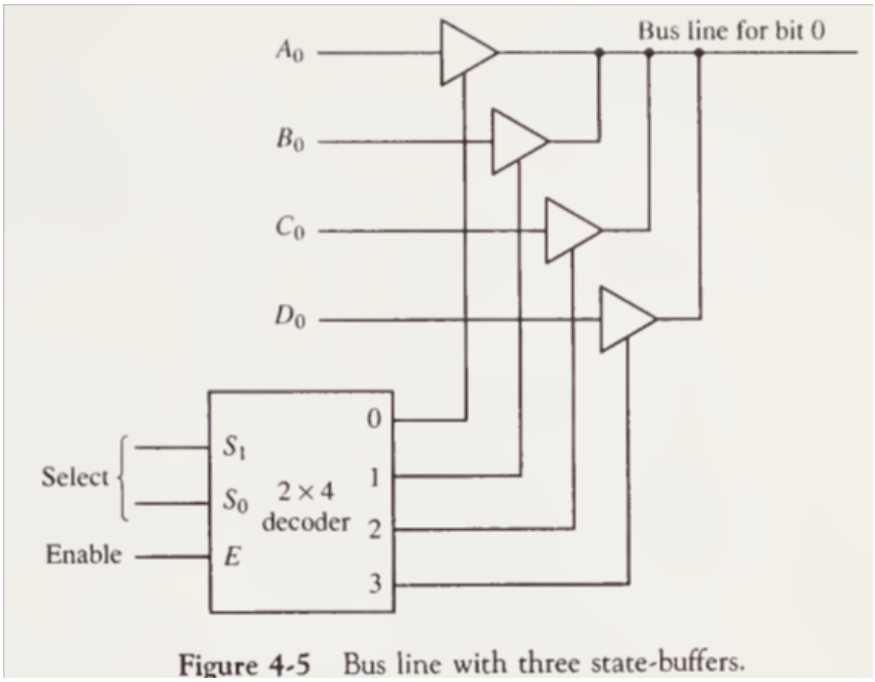
$$\boxed{Z = \text{high impedance} \approx \text{open circuit}}$$

$$C = 1 \Rightarrow Y = A, \qquad C = 0 \Rightarrow Y = Z$$

A decoder can ensure that exactly one register's buffers are enabled at a time.

WHITEBOARD

Corresponds to Mano, Fig. 4-5.



$$\boxed{\text{Only one source may drive the bus at one time.}}$$

All other outputs:

$$\boxed{Z}$$

Worked Example — Textbook Problem 4-5: Building the Bus with Three-State Buffers

Q: Draw a bus system similar to Fig. 4-3 using three-state buffers and a decoder instead of multiplexers.

A: For each bit position, provide one three-state buffer for each register. A 2-to-4 decoder enables exactly one register's buffer set. The same structure is repeated for all four bit positions.

WHITEBOARD

S_1	S_0	Enabled register
0	0	A
0	1	B
1	0	C
1	1	D

For each bit i:

$$A_i, B_i, C_i, D_i \rightarrow \text{four three-state buffers} \rightarrow BUS_i$$

For four 4-bit registers:

$$\boxed{4 \times 4 = 16 \text{ three-state buffers}}$$

3.3 Memory Transfers

Let AR be the address register and DR the data register. A memory read is written

$$DR \leftarrow M[AR],$$

and a memory write is written

$$M[AR] \leftarrow R1.$$

WHITEBOARD

$$\boxed{\text{Read: } DR \leftarrow M[AR]}$$

$$\boxed{\text{Write: } M[AR] \leftarrow R1}$$

$M[AR]$ = memory word selected by the address in AR

---

4 Arithmetic Microoperations

The microoperations most often encountered in digital computers fall into four classes.

WHITEBOARD

Microoperations

1. Register transfer
2. Arithmetic
3. Logic
4. Shift

register transfer: information unchanged

arithmetic/logic/shift: information transformed

Basic arithmetic microoperations include add, subtract, complement, increment, and decrement.

WHITEBOARD

$R3 \leftarrow R1 + R2$	Add
$R3 \leftarrow R1 - R2$	Subtract
$R2 \leftarrow \overline{R2}$	1's complement
$R2 \leftarrow \overline{R2} + 1$	2's complement
$R1 \leftarrow R1 + 1$	Increment
$R1 \leftarrow R1 - 1$	Decrement

Subtraction:

$$A - B = A + \overline{B} + 1$$

4.1 Binary Adder

An n-bit binary adder is constructed by cascading n full-adders. The carry output of each stage feeds the carry input of the next higher-order stage.

WHITEBOARD

Corresponds to Mano, Fig. 4-6.

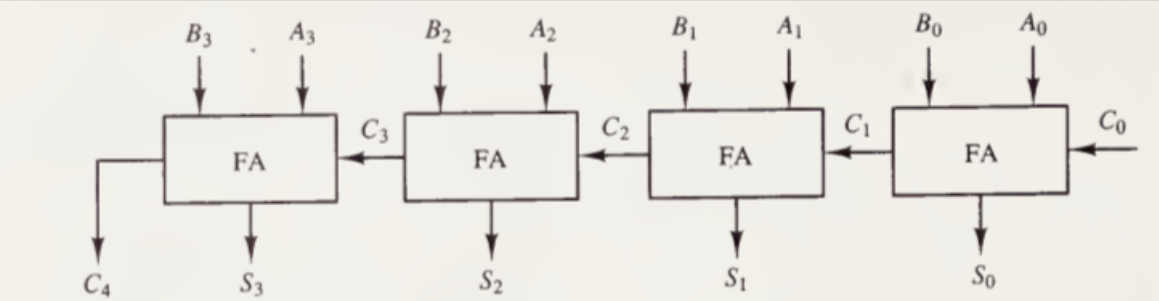


Figure 4-6 4-bit binary adder.

$$\boxed{n\text{-bit adder} = n \text{ full-adders}}$$

Worked Example — Textbook Problem 4-8: Control Logic Plus Arithmetic Addition

Q: Implement

$$x + yz: \qquad AR \leftarrow AR + BR,$$

where AR and BR are n-bit registers and x, y, z are control variables.

A: The plus sign has two different meanings. In the control expression $x + yz$, it means Boolean OR. In the microoperation $AR + BR$, it means arithmetic addition. The adder continuously forms $AR + BR$, while the control function determines whether AR loads the result.

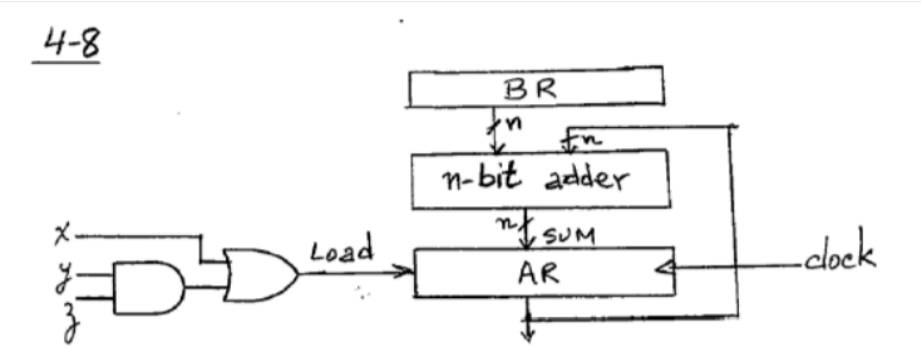
WHITEBOARD

$$\boxed{P = x + yz}$$

$$G = yz, \qquad P = x + G$$

$$\boxed{SUM = AR + BR}$$

$$\boxed{LD(AR) = x + yz}$$



4.2 Adder-Subtractor

Addition and subtraction can be performed by the same hardware. A mode input M controls XOR gates on the B operand and also supplies the least-significant input carry.

WHITEBOARD

Corresponds to Mano, Fig. 4-7.

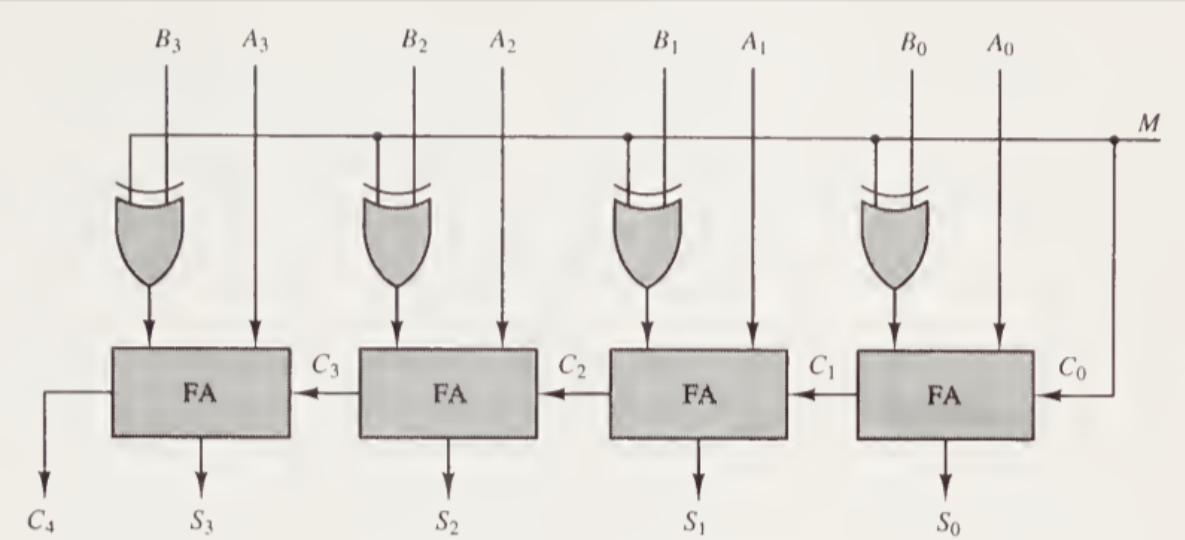


Figure 4-7 4-bit adder-subtractor.

If $M = 0$:

$$B \oplus 0 = B, \qquad C_0 = 0$$

$$\boxed{S = A + B}$$

If $M = 1$:

$$B \oplus 1 = \overline{B}, \qquad C_0 = 1$$

$$S = A + \overline{B} + 1$$

$$\boxed{S = A - B}$$

4.3 Increment

An increment microoperation adds one to a register.

WHITEBOARD

$$\boxed{A \leftarrow A + 1}$$

Examples:

$$0110 + 1 = 0111$$

$$1111 + 1 = 0000, \qquad C_{out} = 1$$

Worked Example — Textbook Problem 4-11: Use the Counter's Built-In Incrementer

Q: Using a 4-bit counter with parallel load and a 4-bit adder, implement

$$x: \quad R_1 \leftarrow R_1 + R_2$$

and

$$x'y: \quad R_1 \leftarrow R_1 + 1,$$

where R_1 is the counter and R_2 is a 4-bit register.

A: Use the external adder only for $R_1 + R_2$. Its output feeds the parallel-load inputs of the counter, and x drives Load. The counter already has increment hardware, so $x'y$ drives Increment directly.

WHITEBOARD

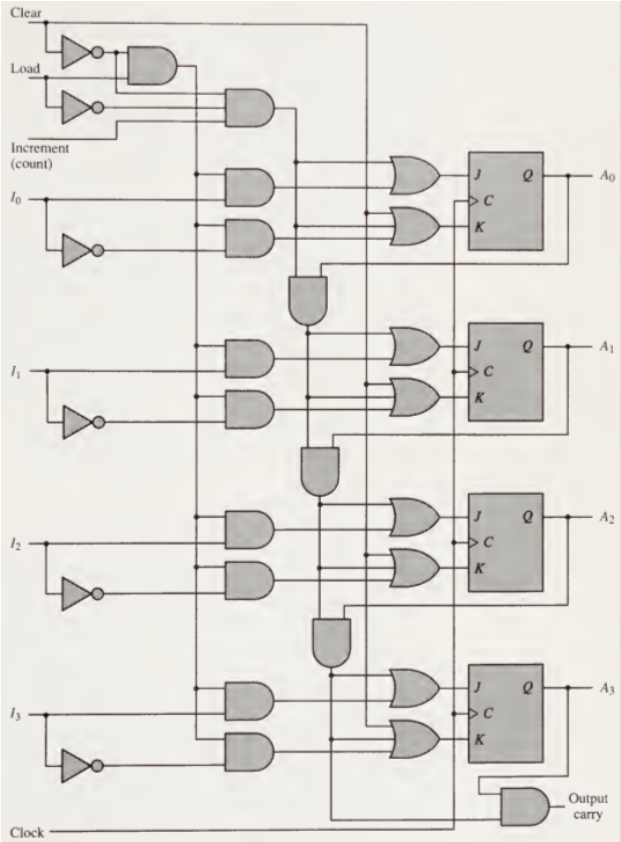


Figure 2-11 4-bit binary counter with parallel load and synchronous clear.

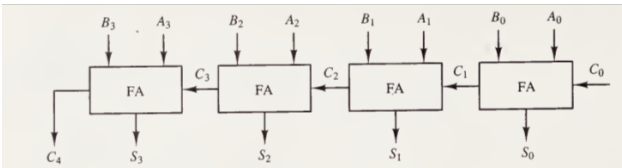


Figure 4-6 4-bit binary adder.

$$\boxed{SUM = R_1 + R_2}$$

$$\boxed{LD = x} \qquad \boxed{INC = x'y} \qquad \boxed{CLR = 0}$$

x	y	LD	INC	Operation
0	0	0	0	No change
0	1	0	1	$R_1 \leftarrow R_1 + 1$
1	0	1	0	$R_1 \leftarrow R_1 + R_2$
1	1	1	0	$R_1 \leftarrow R_1 + R_2$

Worked Example — Register Problem 13: Dedicated Counter Hardware vs. One General Adder

Q: Implement

$$C_1: \quad R_1 \leftarrow R_1 + R_2$$

and

$$C_1'C_2: \quad R_1 \leftarrow R_1 + 1$$

first using a counter with parallel load and then using a 4-bit adder plus external gates.

A: Part (a) follows the same idea as Problem 4-11: exploit the counter's built-in increment operation. Part (b) deliberately uses one adder for both arithmetic operations by controlling the second operand, input carry, and load signal.

WHITEBOARD

Part (a):

$$\boxed{SUM = R_1 + R_2}$$

$$\boxed{LD = C_1} \qquad \boxed{INC = C_1'C_2} \qquad \boxed{CLR = 0}$$

Part (b):

$$\boxed{Y_i = C_1 R_{2i}}, \qquad i = 0, 1, 2, 3$$

$$\boxed{C_{in} = C_1'}$$

$$\boxed{LD = C_1 + C_2}$$

$$\boxed{D = R_1 + Y + C_{in}}$$

C_1	C_2	Y	C_{in}	LD	Operation
0	0	0000	1	0	No change
0	1	0000	1	1	$R_1 \leftarrow R_1 + 1$
1	0	R_2	0	1	$R_1 \leftarrow R_1 + R_2$
1	1	R_2	0	1	$R_1 \leftarrow R_1 + R_2$

Counter design uses dedicated increment hardware.

Adder-only design uses one general arithmetic path.

4.4 Composite Arithmetic Circuit

Mano combines a parallel adder with multiplexers so that the second adder operand Y can be selected from B, $\overline{B}$, 0, or 1. The result is

$$D = A + Y + C_{in}.$$

WHITEBOARD

Corresponds to Mano, Fig. 4-9.

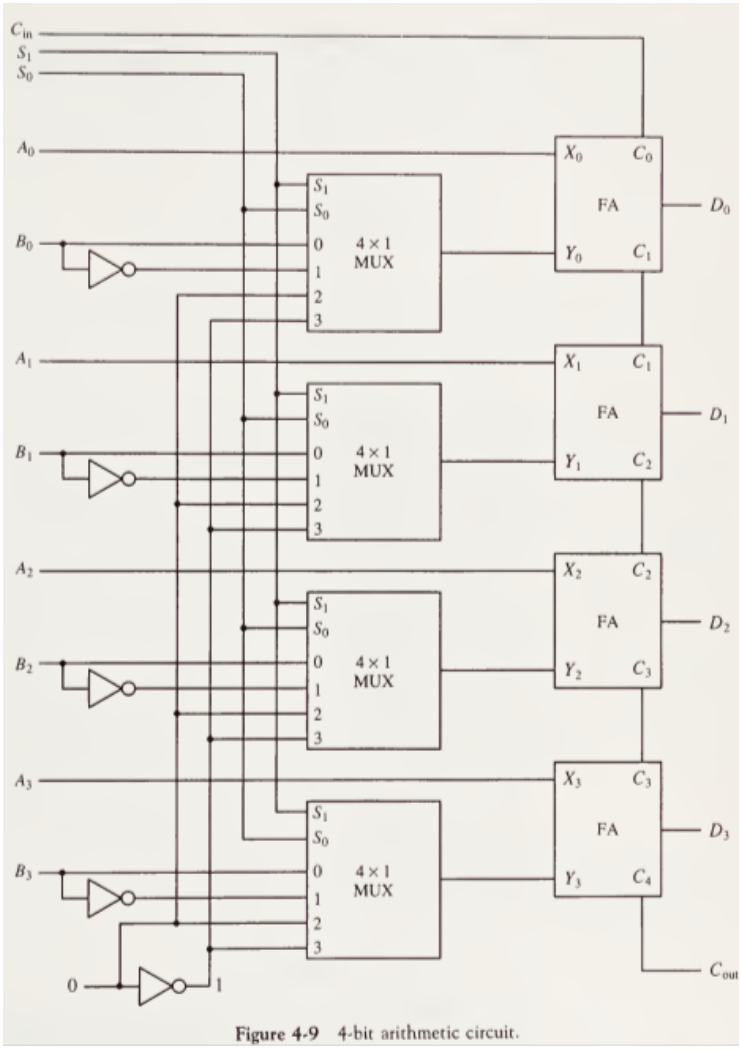


Figure 4-9 4-bit arithmetic circuit.

$$\boxed{D = A + Y + C_{in}}$$

S_1	S_0	Y
0	0	B
0	1	$\overline{B}$
1	0	0
1	1	1

The eight control combinations produce seven distinct arithmetic microoperations.

WHITEBOARD

S_1	S_0	C_{in}	Result
0	0	0	$A + B$
0	0	1	$A + B + 1$
0	1	0	$A - B - 1$
0	1	1	$A - B$
1	0	0	A
1	0	1	$A + 1$
1	1	0	$A - 1$
1	1	1	A

Worked Example — Textbook Problem 4-14: Cascade Two 4-Bit Arithmetic Slices

Q: Assume the 4-bit arithmetic circuit of Fig. 4-9 is one IC package. Connect two such packages to form an 8-bit arithmetic circuit.

A: One package handles bits 3:0 and the other handles bits 7:4. Both receive the same function-selection inputs. The low-order carry-out becomes the high-order carry-in.

WHITEBOARD

$$IC_L: \quad A_3 : A_0,\ B_3 : B_0 \rightarrow D_3 : D_0$$

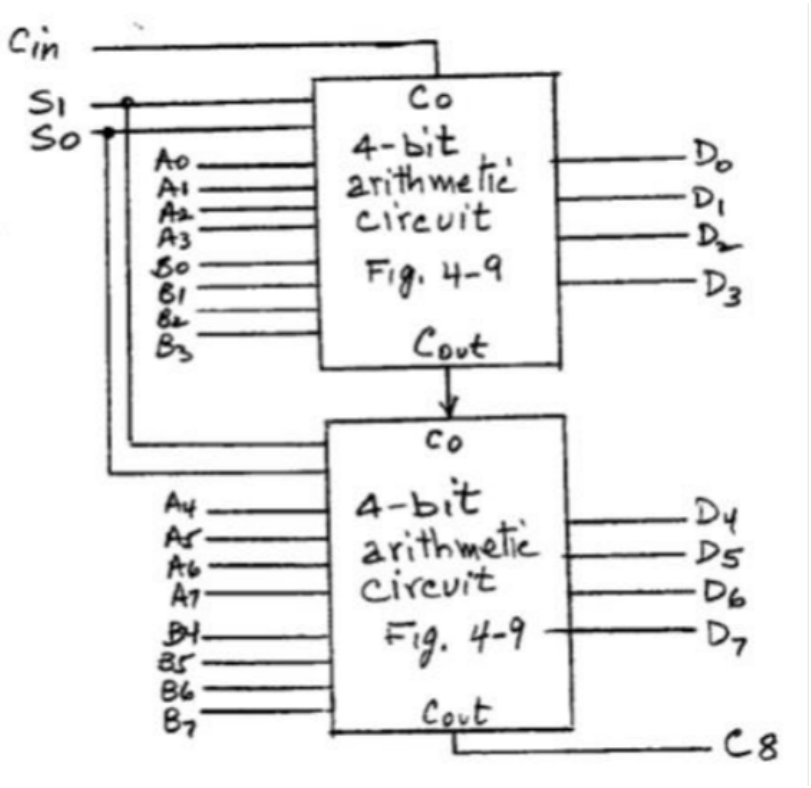
$$IC_H: \quad A_7 : A_4,\ B_7 : B_4 \rightarrow D_7 : D_4$$

$$\boxed{S_1, S_0 \text{ common to both ICs}}$$

$$\boxed{C_{in} \rightarrow IC_L}$$

$$\boxed{C_4 = C_{out}(IC_L) \rightarrow C_{in}(IC_H)}$$

$$\boxed{C_8 = C_{out}(IC_H)}$$



---

5 Logic Microoperations

Logic microoperations perform bit-by-bit operations on register contents.

For example, if

$$R1 = 1010, \qquad R2 = 1100,$$

then

$$R1 \leftarrow R1 \oplus R2$$

produces

$$0110.$$

WHITEBOARD

---

logic microoperations act bit-by-bit

$$\begin{array}{r} 1010 \\ \oplus\ 1100 \\ \hline 0110 \end{array}$$

5.1 Arithmetic Plus vs. Boolean OR

The symbol + has different meanings depending on context. In a Boolean control function it denotes OR; inside an arithmetic register microoperation it denotes addition. For register logic OR, Mano uses $\vee$.

WHITEBOARD

$$P + Q: \quad R1 \leftarrow R2 + R3$$

Here:

$$\boxed{P + Q = \text{Boolean OR}}$$

$$\boxed{R2 + R3 = \text{arithmetic addition}}$$

For register logic OR:

$$\boxed{R4 \leftarrow R5 \vee R6}$$

5.2 Sixteen Two-Variable Logic Functions

Two binary variables define

$$2^{2^2} = 16$$

possible Boolean functions. Computers commonly provide a smaller useful set such as AND, OR, XOR, and complement.

WHITEBOARD

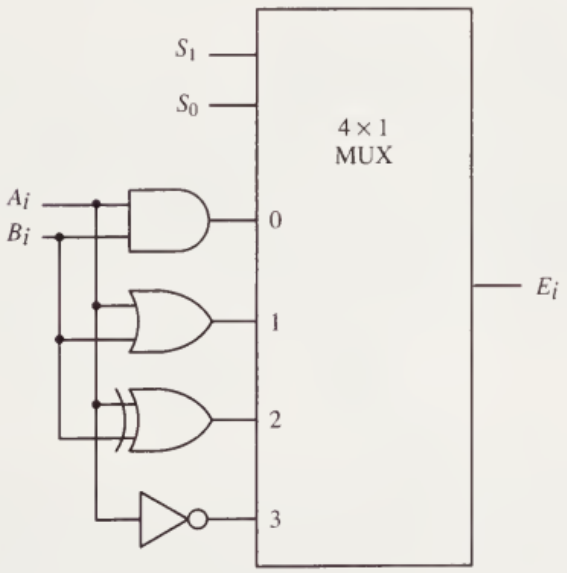
$$\boxed{2^{2^2} = 16 \text{ possible two-variable logic functions}}$$

Common basic set:

$$\boxed{AND, \quad OR, \quad XOR, \quad complement}$$

Corresponds to Mano, Fig. 4-10.

Figure 4-10 One stage of logic circuit.



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \overline{A}$	Complement

(b) Function table

S_1	S_0	Operation
0	0	AND
0	1	OR
1	0	XOR
1	1	Complement A

Worked Example — Textbook Problem 4-16: Generate Any of the 16 Logic Functions

Q: Derive a combinational circuit that selects and generates any of the 16 logic functions listed in Table 4-5.

A: A two-variable Boolean function is determined by the four outputs corresponding to input combinations $00, 01, 10, 11$. Treat those four truth-table values as the data inputs of a 4-to-1 multiplexer. The variables x and y select which truth-table entry appears at the output.

WHITEBOARD

Corresponds to Mano, Table 4-5.

TABLE 4-5 Truth Tables for 16 Functions of Two Variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

$$xy = 00 \Rightarrow F = S_3$$

$$xy = 01 \Rightarrow F = S_2$$

$$xy = 10 \Rightarrow F = S_1$$

$$xy = 11 \Rightarrow F = S_0$$

$$\boxed{F = x'y'S_3 + x'yS_2 + xy'S_1 + xyS_0}$$

$$\boxed{I_0 = S_3, \quad I_1 = S_2, \quad I_2 = S_1, \quad I_3 = S_0}$$

5.3 Applications of Logic Microoperations

Logic microoperations can selectively set, complement, clear, mask, insert, or compare bit positions.

WHITEBOARD

Selective set:

$$\boxed{A \leftarrow A \vee B}$$

$$1010 \vee 1100 = 1110$$

Selective complement:

$$\boxed{A \leftarrow A \oplus B}$$

$$1010 \oplus 1100 = 0110$$

Selective clear:

$$\boxed{A \leftarrow A \wedge \overline{B}}$$

Mask:

$$\boxed{A \leftarrow A \wedge B}$$

Equality test:

$$\boxed{A = B \iff A \oplus B = 0}$$

Insert is a two-step operation: mask the target field, then OR in the replacement bits.

WHITEBOARD

$$0110\,1010 \wedge 0000\,1111 = 0000\,1010$$

then

$$0000\,1010 \vee 1001\,0000 = \boxed{1001\,1010}$$

$$\boxed{\text{Insert} = \text{Mask then OR}}$$

Worked Example — Textbook Problem 4-18: Choose the Logic Mask

Q: Register A contains 11011001. Determine the operand B and logic microoperation required to obtain (a) 01101101 and (b) 11111101.

A: Part (a) requires selected bits to flip, so use XOR. The required mask is the XOR of the initial and desired values. Part (b) only requires selected positions to be forced to 1, so use OR.

WHITEBOARD

$$A = 11011001$$

Part (a):

$$B = A \oplus A_{desired}$$

$$11011001 \oplus 01101101 = 10110100$$

$$\boxed{B = 10110100} \qquad \boxed{A \leftarrow A \oplus B}$$

Check:

$$11011001 \oplus 10110100 = \boxed{01101101}$$

Part (b):

$$\boxed{B = 11111101} \qquad \boxed{A \leftarrow A \vee B}$$

$$11011001 \vee 11111101 = \boxed{11111101}$$

Worked Example — Register Problem 11: Design Masks for a 16-Bit Word

Q: Given

$$A = 00001111\ 10101010,$$

find the operation and operand needed to (a) clear all even bit positions, (b) set the leftmost four bits, and (c) complement the center eight bits.

A: Use AND with zeros to clear selected positions, OR with ones to set selected positions, and XOR with ones to complement selected positions.

WHITEBOARD

$$A = 00001111\ 10101010$$

(a) Clear even positions:

$$\boxed{B = 10101010\ 10101010}$$

$$A \wedge B = \boxed{00001010\ 10101010}$$

(b) Set leftmost four bits:

$$\boxed{B = 11110000\ 00000000}$$

$$A \vee B = \boxed{11111111\ 10101010}$$

(c) Complement center eight bits:

$$\boxed{B = 00001111\ 11110000}$$

$$A \oplus B = \boxed{00000000\ 01011010}$$

Task	Operation
Clear selected bits	AND
Set selected bits	OR
Complement selected bits	XOR

---

6 Shift Microoperations

Shift microoperations move register bits to the left or right. Mano distinguishes logical, circular, and arithmetic shifts.

WHITEBOARD

Shift Microoperations

logical circular arithmetic

$R \leftarrow shl\ R$	logical shift left
$R \leftarrow shr\ R$	logical shift right
$R \leftarrow cil\ R$	circular shift left
$R \leftarrow cir\ R$	circular shift right
$R \leftarrow ashl\ R$	arithmetic shift left
$R \leftarrow ashr\ R$	arithmetic shift right

6.1 Logical and Circular Shifts

A logical shift inserts 0 into the vacant position. A circular shift feeds the bit leaving one end back into the other end.

WHITEBOARD

Logical:

$$1011 \xrightarrow{shr} 0101$$

$$1011 \xrightarrow{shl} 0110$$

$$\boxed{\text{logical shift inserts 0}}$$

Circular:

$$1011 \xrightarrow{cir} 1101$$

$$1011 \xrightarrow{cil} 0111$$

$$\boxed{\text{circular shift loses no bit}}$$

6.2 Arithmetic Shifts

For signed 2's-complement data, an arithmetic shift right preserves the sign by copying the sign bit into the vacated high-order position. An arithmetic shift left inserts 0 at the low-order end and corresponds to multiplication by 2, but it may overflow.

WHITEBOARD

$$\boxed{ashr \Rightarrow \text{divide by 2 while preserving sign}}$$

$$\boxed{ashl \Rightarrow \text{multiply by 2}}$$

Arithmetic-left overflow test:

$$\boxed{V_s = R_{n-1} \oplus R_{n-2}}$$

$$V_s = 1 \Rightarrow \boxed{\text{overflow}}$$

Worked Example — Textbook Problem 4-20: Arithmetic Shift and Overflow

Q: An 8-bit register contains 10011100. Find the result after an arithmetic shift right. Starting again from the original value, find the result after an arithmetic shift left and determine whether overflow occurs.

A: The arithmetic right shift copies the sign bit. The arithmetic left shift inserts 0 at the right and tests the two original most significant bits for overflow.

WHITEBOARD

$$R = 10011100$$

$$\text{ashr}(10011100) = \boxed{11001110}$$

$$\text{ashl}(10011100) = \boxed{00111000}$$

$$V_s = R_7 \oplus R_6 = 1 \oplus 0 = 1$$

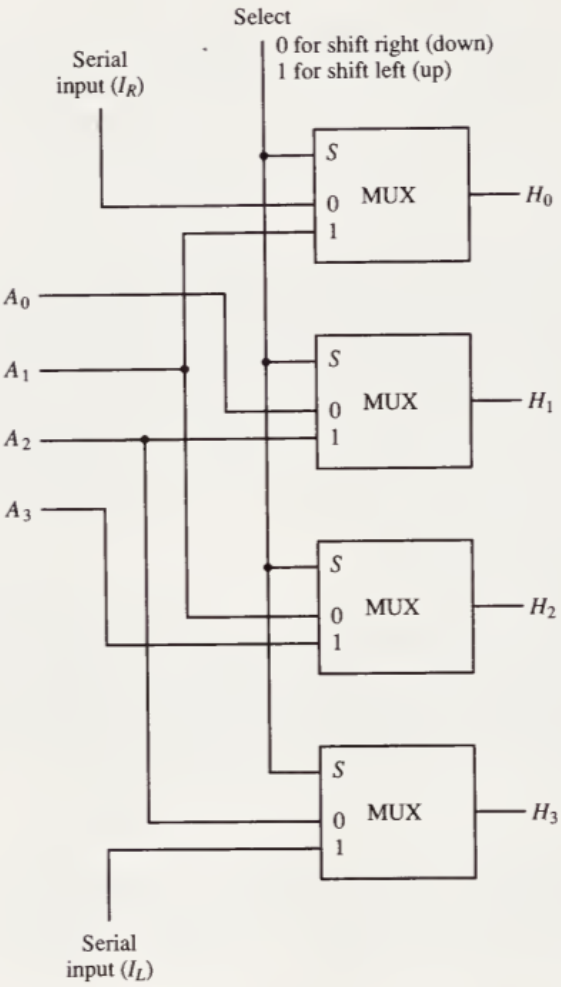
$$\boxed{\text{Overflow occurs}}$$

6.3 Combinational Shifter

A processor can implement shifting as a combinational datapath operation instead of requiring a separate shift-register clock cycle. Mano's 4-bit combinational shifter uses multiplexers to select the source for each output bit.

WHITEBOARD

Corresponds to Mano, Fig. 4-12.



Function table

Select	Output			
S	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L

Figure 4-12 4-bit combinational circuit shifter.

$\boxed{S = 0 \Rightarrow \text{shift right}}$

$\boxed{S = 1 \Rightarrow \text{shift left}}$

Worked Example — Register Problem 12: Johnson Counter as a Shift-Register Application

Q: A switch-tail ring counter (Johnson counter) feeds the complement of the serial output of a right-shift register back into the serial input. Starting from 0000, list the sequence until the register returns to 0000. How many states does an n-bit Johnson counter have in its normal sequence?

A: For a right shift, the rightmost bit is the serial output. Its complement enters the leftmost position. Starting from all zeros, ones enter one at a time until the register becomes all ones; then zeros enter one at a time until the register returns to all zeros.

WHITEBOARD

$$Q_3^+ = Q_0', \qquad Q_2^+ = Q_3, \qquad Q_1^+ = Q_2, \qquad Q_0^+ = Q_1$$

$$\boxed{0000 \to 1000 \to 1100 \to 1110 \to 1111 \to 0111 \to 0011 \to 0001 \to 0000}$$

$$\boxed{\#\text{ states} = 2n}$$

---

7 Arithmetic Logic Shift Unit

Rather than provide separate arithmetic and logic hardware for every register, computers route source-register data through a shared arithmetic logic unit (ALU). The selected operation is performed combinationally, and the result is transferred to a destination register.

WHITEBOARD

$$\boxed{\text{ALU} = \text{Arithmetic Logic Unit}}$$

$$\boxed{\text{source registers} \to \text{ALU} \to \text{destination register}}$$

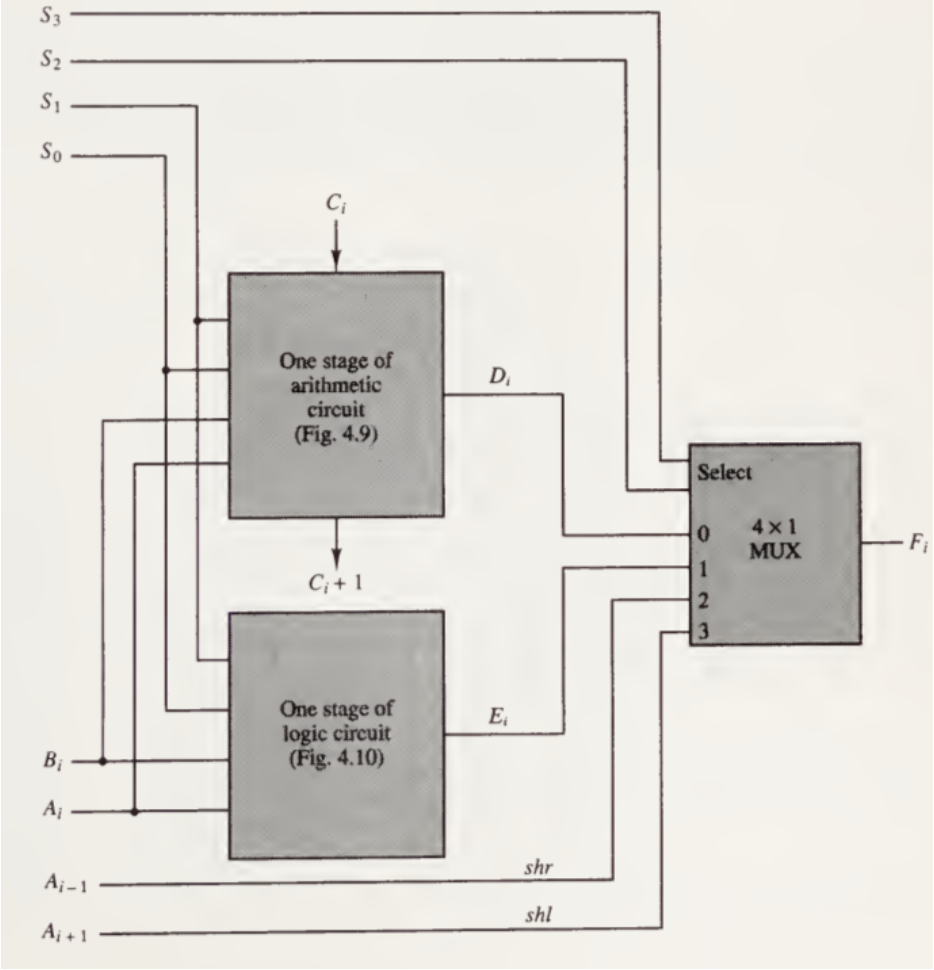
$$\boxed{\text{ALU is combinational}}$$

The arithmetic, logic, and shift circuits developed earlier can be combined into one arithmetic logic shift unit.

WHITEBOARD

Corresponds to Mano, Fig. 4-13.

Figure 4-13 One stage of arithmetic logic shift unit.



Arithmetic + Logic + Shift

→ one selectable operational unit

For an n-bit datapath, the one-bit stage is repeated n times. The high-order selection bits choose the overall operation class.

WHITEBOARD

---

n-bit unit = n repeated stages

S_3	S_2	Operation class
0	0	Arithmetic
0	1	Logic
1	0	Shift right
1	1	Shift left

The complete control table is the final reference for the chapter.

WHITEBOARD

Corresponds to Mano, Table 4-8.

TABLE 4-8 Function Table for Arithmetic Logic Shift Unit

Operation select						
S_3	S_2	S_1	S_0	C_{in}	Operation	Function
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	×	$F = A \wedge B$	AND
0	1	0	1	×	$F = A \vee B$	OR
0	1	1	0	×	$F = A \oplus B$	XOR
0	1	1	1	×	$F = \overline{A}$	Complement A
1	0	×	×	×	$F =$ shr A	Shift right A into F
1	1	×	×	×	$F =$ shl A	Shift left A into F

one datapath + control selection = many microoperations

8 Chapter Synthesis

Chapter 4 ties together the storage components of earlier chapters with the datapath operations used inside a computer. Registers hold information, RTL describes transfers and transformations, common buses move data among modules, and arithmetic, logic, and shift hardware perform the selected microoperations.

The worked problems reinforce an important design principle: the symbolic microoperation tells us what must happen, while the hardware organization determines how efficiently it can be implemented.

WHITEBOARD

registers → RTL → controlled transfers

→ common bus → memory transfer

→ arithmetic + logic + shift

→ ALU

Registers store data.

Microoperations transfer or transform it.

Control determines what happens and when.

Good hardware design reuses existing functional units.
